

ICS 3.0 — CTF Write-ups

How I actually solved them, one at a time

Pasidu Mihiranga

Seventeen challenges — crypto, web, pwn, reversing, forensics, cloud

A collected record of the boxes I broke during ICS 3.0, written the way I'd explain them to a teammate over coffee. Nothing here is theory — every flag below actually came off the scoreboard.

Before we start

I kept notes as I went, and this is me cleaning them up into one place. I've tried to write down not just *what* the answer was but the order my head actually moved in — the dead ends I walked into, the moment something clicked, the little "oh, that's what the flavour text meant" beats. A few of these took me all evening; a couple fell over in ten minutes once I saw the trick.

Where a challenge needed a real script to finish, I haven't pasted the code into this document — it clutters the read and you can't run a PDF anyway. Instead there's a small arrow line under the solution pointing at the exact file in the repo. The scripts live under `ALLWriteUps/scripts/` unless noted otherwise.

The event used two flag prefixes, `ICS{}` and `iCS{}` (lowercase *i*), and yes — that difference is load-bearing on at least one challenge. I've written each flag exactly as the scoreboard accepted it.

Contents

Before we start	1
1 House of Mirrors	2
2 BOS-400 Telegraph Archive	2
3 The Man on the Sealed Train	2
4 The Seduction	3
5 Invitation to Tea Party	4
6 Chameleo Great Power	4
7 Kotoamatsukami	5
8 Vanishing Stars	5
9 Man Behind the Mask	5
10 The Leader	6
11 The Art of Programming	6
12 Fairchild	7
13 The Contaminated Container	7
14 The Seventh Encore	7
15 The Candidate Who Never Whispered	8
16 The Impossible Meeting	8
17 The Document That Remembers	9
Scoreboard	10

1. House of Mirrors

[crypto]

Two files: `encrypt.py` and `output.txt`. The output just hands you `n`, `e = 3`, and a ciphertext `c`. Reading the encrypt script, the flag goes through four layers on the way out:

`flag` → XOR (LCG keystream) → ARC4 → `bytes_to_long` → RSA ($e=3$).

The thing that jumped out immediately is `e = 3` with no padding. The flag is short, so the plaintext integer `m` is small enough that $m^3 < n$ — which means RSA never actually wraps around the modulus. The ciphertext is literally m^3 . No factoring, no fancy attack: just take an integer cube root and you've got `m` back. I used a binary-search cube root so I didn't have to trust floating point.

From there it's all deterministic and the keys are sitting in the source. The ARC4 key is `sha256("Unlucky13")[:16]` — the "Unlucky 13" theme is a hint, but honestly the string is right there in the file. ARC4 is symmetric so decrypting is the same op. The last peel is the XOR layer: a little LCG PRNG seeded with 13 (`state = (1664525*state + 1013904223) mod 232`, one byte each step). Regenerate the keystream, XOR again, done.

→ Solver script: `house_of_mirrors_solve.py`

FLAG `iCS{m1rr0r_m1rr0r_0n_th3_w4ll_3ncrypt10n_15_th3_k3y_t0_4ll}`

2. BOS-400 Telegraph Archive

[crypto / forensics]

Two artefacts: a `challenge.txt` with a "BOS-400" byte stream, and `bos_code_card.png` holding an eight-glyph verification key. The card reads `M7KPX9ZL`, and the archive notes say a 1987 fragment `C0D3` came right after it — so the full authentication material is `M7KPX9ZLC0D3`.

The "256-bit member of the seal standardised in FIPS PUB 180-4" is a wordy way of saying SHA-256. Hash that string, keep the leftmost 16 bytes, and that's the fingerprint that gets repeated across the message.

The ledger describes the *forward* process: characters become codebook addresses, XOR with the repeated hash, then a high/low nibble swap on every byte. So to undo it I just run the inverse in reverse — swap nibbles first (a nibble swap is its own inverse), XOR with the same fingerprint, and read the bytes back through the codebook. That gives `>S8lKW0;iCS{b0s400_d3c0mm1ss10n3d}`. The note says the terminal jams exactly eight characters of sync chatter in front of every payload, so I drop the first eight and I'm left with the real thing.

→ Solver script: `bos_code_card_solve.py`

FLAG `iCS{b0s400_d3c0mm1ss10n3d}`

3. The Man on the Sealed Train

[crypto / multi-stage]

This one is a proper chain — six stages, each stage coughing up the key for the next. The whole thing is Lenin's April 1917 journey from Zürich to Petrograd dressed up in deliberately archaic language, and every "key" is a real fact from that trip.

1. **A1Z26** on `childhood_letter.txt`: the numbers 18-5-22-15-12-21-20-9-15-14 spell **REVOLUTION**.
2. **Vigenère** on `prisoner_note.txt` with that key gives SIBERIAN EXILE CREATED THEORY. This one's just confirmation — nothing downstream eats its output, which is worth knowing so you don't waste time looking for a use for it.
3. **Caesar** on `exile_diary.txt`. The shift isn't a Vigenère key — the note says "the subject's *measure* marks the distance of exile", and the index term is **CAPITALISM**, whose *length* is 10. Shift back by 10 and the diary points at a six-letter German city on a lake starting with Z: **Zürich** (spelled ZURICH — accents are lost at the border, per the footnote).
4. **ZipCrypto** on `newspaper_clipping.zip` with ZURICH. Inside is an MD5 digest and a city list — brute the eight cities, and `md5("PETROGRAD")` matches.
5. **ZipCrypto** again on the manifest with PETROGRAD. This gives the sealing instructions.
6. **SHA-256 counter-mode XOR**. The prose is a spec once you translate it: `seed = PETROGRAD|SEALED TRAIN|` followed by the old-style date as four raw bytes (New Style 16-04-1917 minus 13 Julian days = 03-04-1917 → bytes 03 04 13 11). Keystream is `sha256(seed || big-endian counter)` concatenated; XOR against the ciphertext.

The one genuine trap: the manifest has *two* route strings. Only the one literally labelled "Route note" (**SEALED TRAIN**) decrypts to 285/285 printable bytes; the other gives garbage. Score by printability and it decides itself. The decrypted speech ends in a hex archive mark that decodes to the flag — and note the **lowercase i**. The first byte is 0x69, not 0x49. Submitting `ICS{...}` gets rejected; you have to send it exactly as it decoded.

→ Solver script: [sealed_train_solve.py](#)

FLAG `iCS{POW3R_W45_1N_7H3_5P33CH}`

4. The Seduction

[crypto / audio stego]

A 42-second stereo WAV. First thing I did was look at the raw bytes, and the LIST/INFO metadata handed me an AES-256-CTR payload — method, KDF (SHA-256), an IV, and the ciphertext. So the audio has to hide the passphrase.

The clue points at frequency hopping (a nod to Hedy Lamarr). The normal stereo mix is a decoy; the real signal is the side channel, $L - R$, and subtracting the channels isolates a clean hopping burst between 12.0 s and 26.4 s. It changes frequency every 0.12 s, and the eight tones pair up into four binary FSK pairs — within the active pair, lower tone is a 0 and upper is a 1. Reading 120 bits MSB-first gives a 55 **aa** preamble followed by **VELVET STATIC**.

The metadata told me to SHA-256 that phrase for the AES key. Decrypt the CTR stream and out comes the message plus the flag.

I'll be honest — I burned a good while down the wrong rabbit hole here first, trying to demodulate the FSK into base-3, Morse, tap code, everything, before I stepped back and realised the phrase itself was the key material. Lesson filed.

→ Solver script: [the_seduction_solve.py](#)

FLAG `ICS{v3lv3t_st4t1c_h0ps_3142_7c9e}`

5. Invitation to Tea Party

[forensics / crypto]

A Google Drive link to a 153 MB `tea_party.zip`. The size is a warning that you shouldn't just unpack blindly, so I read the ZIP central directory first. It holds 1755 JPEGs across 39 subdirectories (45 images each) plus one odd file, `drink_me.txt`. And the subdirectory names are exactly a flag alphabet: 0-9, A-Z, and `_ { }`. Nobody names 39 folders like that by accident — the tree is literally called `alice_tea_table_cipher`, so it's a substitution table where each folder is a plaintext character and the images inside are its symbols.

`drink_me.txt` is 2624 hex characters. The bait: the JPEG filenames are 16 hex chars each, which screams "slice the ciphertext into 16-char tokens and look up filenames" — and that resolves exactly zero of them. The real move is to hash the image *contents*. Sweeping algorithms and both ends of each digest, SHA-256 hits on both the first *and* last 16 chars of the same files, which only happens if the whole 64-char digest is in the blob. So the record size is 64, not 16: $2624/64 = 41$ characters.

From there it's one lookup per record: `sha256(image bytes)` maps to the folder name. The output opens with the ICS prefix and closes with a brace, which is its own proof it's right. (Fun detail: one digest appears twice because the letter T repeats and the encoder happened to reuse the same image.)

→ Solver script: `tea_party_solve.py`

```
FLAG ICS{W3LCOME_TO_TH3_M4D_H4TT3RS_T34_P4RTY}
```

6. Chameleo Great Power

[web]

Start page is `/culture` for "Chameleo International", and the brief says find the secret "where the power is at its peak". The nav shows `/team` and `/login`, but digging through the client bundle leaks a pile of hidden routes — including `/ceo/dashboard` and `/ceo/employees/search`. The "power at its peak" line is basically telling you to aim at the CEO routes.

The chain went like this. `/team/manduka-rapatulo` gives me a profile and, crucially, the portal ID format (`firstname.lastname`) → username `manduka.rapatulo`. The `/recover` page asks "what is your pet's name?", and a bit of guessing at common pet names lands on `Milo`, which drops me a valid session cookie as `Manduka`.

`Manduka`'s dashboard shows a *disabled* "System Users" card — but the restriction is only in the UI. Hitting `/users` directly with the session works and leaks password hashes. The CEO `rango.jayamaha` has hash `5f4dcc3b5aa765d61d8327deb882cf99`, which is the world's most famous MD5: `password`. Log in as CEO.

The CEO's `/ceo/employees` page has an HTML comment admitting the directory still accepts "selectors from the retired archive service". The search endpoint's `fields` parameter is injectable — `fields=sqlite_version()` returns `3.51.3`, confirming SQL selector injection. Query `sqlite_master`, find a hidden `confidential_notes` table, read it, and the note is the flag.

```
FLAG iCS{s3l3c7_qu3r13s_n33d_b0und4r13s}
```

7. Kotoamatsukami

[web / logic]

Two endpoints. `/api/seal` checks that your JSON directive is benign (`action=request_access`, `subject=guest`, `level=public`) and, if so, signs it with HMAC-SHA256. `/api/execute` verifies the signature and then *acts* on the payload.

The bug is a classic parser-discrepancy on duplicate keys: the signing gateway keeps the **first** occurrence of a repeated key, the execution engine keeps the **last**. So I hand it a payload with both:

```
action:request_access, action:grant_access, subject:guest, level:public, level:level-5
```

The gateway reads it as `request_access / public` — all allowed — and happily signs it. The execute endpoint reads the *same bytes* as `grant_access / level-5`, and since the signature is genuinely valid it carries out the elevated action and hands back a `decision_session` cookie. Use that cookie on `/api/archive` and the restricted archive opens up.

```
FLAG ICS{7h3_d3c1510n_w45_n3v3r_y0ur5}
```

8. Vanishing Stars

[cloud]

SkyTrace Aerospace says its satellite SVT-7 was lost after launch. An engineer disagrees. The site is a mission dashboard; the `/telemetry` page already smells wrong — the last "public" packet is NOMINAL, then nine minutes later the status flips to FAILED and *then* the stream "terminates". The termination follows the status change, not a real signal loss.

`robots.txt` disallows `/storage/skytrace-public/deployment-backup.zip` — which of course is the first place I looked. That ZIP is the whole Terraform config and source. Two gifts inside: a leaked function key (`skytrace-public-processor-v1`) in `old.env.example`, and a path traversal in `telemetry_processor.py` — it checks `startswith("public/")` *before* normalising the path, so `public/../../anything` sails through. Classic check-before-normalise TOCTOU.

With the function key and the traversal I can read the private telemetry bucket. Then the IAM layer over-shares: `telemetry-processor-role` can assume `archive-auditor` with no restriction, so I mint an STS token for it, list `skytrace-mission-archive`, and read the production logs. The logs are the smoking gun — packets *kept arriving* across orbits 2 and 3 after the "failure", and an executive override at 18:20 UTC flipped the status manually. `executive-decision.txt` spells it out: the satellite is fine, keep publishing the failure story, the insurance claim needs it to look unrecoverable. \$47M of fraud, and the flag is in that memo.

```
FLAG ICS{7H3_5473LL173_N3V3R_F41L3D}
```

9. Man Behind the Mask

[pwn]

One stripped x86-64 PIE binary, `redemption_core`. Canary, NX, PIE all on — so no shellcode-on-stack, this is a leak-then-ROP job. Strings tease `lord_ruin_script.txt` (the file I want to read) and a stray blob `NIURKSAMH9`.

Reading the disassembly, `main` calls three functions:

- `interview()` does `printf(buf)` on my input with no format string of its own — a textbook format-string bug, and it runs *first*.

- `validate_identity()` reads 0x100 bytes into an 0x50-byte struct — a clean stack overflow. It also checks a role field equals RUIN and a key field equals the 64-bit immediate that decodes to NIURKSAM.
- A third, never-called function that opens the script file but only ever `write()`s one byte — a decoy. I ignore it and build my own chain.

So: use the format string to leak the stack canary (index 23, ends in 00) and a PIE-mapped return address (index 25, minus 0x15cf gives the page-aligned base). Then in `validate_identity` I lay down the filler, put RUIN and NIURKSAM in their fields, replant the exact leaked canary so `__stack_chk_fail` doesn't fire, and from the return address onward I ROP: `open("lord_ruin_script.txt")` → `read(3, bss, 0x100)` → `write(1, bss, 0x100)`. That dumps the script and the flag with it.

→ Solver script: [Man_Behind_the_Mask_exploit.py](#)

FLAG `ICS{N3W_M45K_54M3_M3M0RY}`

10. The Leader

[reversing]

A remote `nc` service plus a stripped, statically linked ELF. It asks for four "archive fields" (ALPHA-DELTA) and has a `TracerPid` anti-debug check, so I did this mostly static.

- **ALPHA**: reads an int, formats it back to decimal, DJB2-hashes the string (`h = h*33 + byte`, seed 0x1505) and compares to 0x7c78cdd3. Inverting over decimal strings gives 1337.
- **BRAVO**: a 10-char string, compared case-insensitively byte by byte → `c0rrupt10n`.
- **CHARLIE**: a tiny stack VM in `.rodata` computing `((x XOR 0x0f01) + 0x1a) == 0x172b` → 1984.
- **DELTA**: a 16-bit byte-mixing transform that must equal 0xbbb7; brute all 16-bit inputs and only 1776 works. (It also re-checks `/proc/self/status` for a tracer.)

Feed 1337 / `c0rrupt10n` / 1984 / 1776 and the terminal assembles the key and prints the flag. The four values are the story — 1984, corruption, 1776, 1337 — and they double as the decryption key the binary uses internally.

FLAG `ICS{1984_c0rrupt10n_1776_1337}`

11. The Art of Programming

[reversing / esolang]

The "abstract painting" they give you isn't abstract at all — the border uses all 20 colours of the **Piet** esoteric language, which is designed to look like a Mondrian. Each codel is a 10×10 block, so the 600×600 image is really a 60×60 Piet program.

Running it with the normal initial state (DP right, CC left, empty stack) prints `ICS{This_is_Not_the_flag}` — which is obviously a decoy telling you so to your face. The border actually weaves *two* execution lanes together. Selecting the other lane (codel chooser to the right) and seeding the stack with ASCII `_a` (95, 97) runs the hidden lane and prints the real one.

FLAG `ICS{This_is_Mondarian_art}`

12. Fairchild

[stego / OSINT]

`fairchild.zip` gives you `stratofortress.pdf` — except the magic bytes are `89 50 4E 47`, so it's a PNG, not a PDF. The extension is the first lie. The image is a seven-line poem, and each poem line is a *slot* in a hidden string; each slot's character is stashed in a different corner of the file, and every payload carries its own poem line as a join key so the rendered image *is* the ordering.

I walked the PNG chunk table by hand, because everything valuable lives exactly where a compliant decoder stops looking: an `eXIf` comment, a `zTXt` chunk, 560 bytes past `IEND`, an appended ZIP, LSB stego, and a completely made-up `quTx` private chunk. Each fragment has the form `*<char>*`, its own MD5, and the poem line. Assembled in poem order they spell **FH-227D** — the Fairchild Hiller FH-227D, the aircraft of Uruguayan Air Force Flight 571, the 1972 Andes crash.

But `ICS{FH-227D}` gets rejected. The second hint — "a transformation, not a translation" — is the standard idiom for "hash it, don't decode it", and the whole file uses MD5 as its verification primitive (slot 1 even carries `md5("")`, an empty hash waiting for input). So the answer is `md5("FH-227D")`.

→ Solver script: `fairchild_solve.py`

FLAG `ICS{d2f8c2624be366cdfe3cee06dd216cc3}`

13. The Contaminated Container

[devops / container forensics]

They ship a `docker save` export, `deleted-container-image.tar`. The brief basically tells you the trick: recovery material was copied in during the build and "deleted" before publication. In Docker, deleting a file in a later layer just creates a `.wh.` whiteout in the overlay — the original bytes are still sitting in the earlier layer.

So I extracted every `layer.tar` separately. `find . -name ".wh.*"` points at `.wh.fragment-a.txt`; the actual `fragment-a.txt` is alive in an earlier layer and reads `1m4g3_14y3r5_` (Fragment A). Fragment B comes from the image config JSON's history: an `ENV RECOVERY_FRAGMENT_B=r3m3mb3r` that was later cleared to empty — but the history remembers every command, so `r3m3mb3r` is right there.

The `/app` layer has `license.dat` and a `maintenance-loader.pyc`. Disassembling the bytecode (Python's own `dis+marshal`, since 3.11+ fights the usual decompilers), the format is a DCLIC-V1 header, a 16-byte CTR nonce, then AES-CTR ciphertext keyed by `sha256(fragmentA + fragmentB)` = `sha256("1m4g3_14y3r5_r3m3mb3r")`. Replicate that and `license.dat` decrypts to a JSON blob with the flag.

→ Solver script: `decrypt.py`

FLAG `ICS{d3l3t3d_fr0m_v13w_n0t_fr0m_h15t0ry}`

14. The Seventh Encore

[supply chain / build forensics]

Great flavour: a dead singer's hologram was approved for six acts, but production rendered seven from the same source commit. If the source didn't change, whatever got in came from dependency resolution — so I stopped reading source and started diffing the two build environments.

The Windows rehearsal produced six acts; the Linux production runner produced seven. Side by side, the Linux install pulled `@orpheus/stage-linux-x64@4.2.3` while Windows got 4.2.2, and only that one package ran a lifecycle script. The registry audit log explains it: the package was published four seconds mid-build using the never-revoked legacy CI token of a maintainer who *died in 2023* (`legacyTokenRevoked: null, status: deceased`). Because the pipeline ran `npm install --package-lock=false`, the caret range `^4.2.0` floated straight onto the poisoned version. Windows never saw it because npm skipped the Linux binary as OS-incompatible — the attack was only reachable on the platform that actually shipped.

The payload is in `calibration.dat`: a 15-byte magic, a 16-byte IV, then AES-256-CTR. The key is `sha256(trustedIntegrity)`, and the integrity hash of the legitimate 4.2.2 release is sitting in the provenance attestation the whole time — the artefact written to prove the malware wasn't there is the key that unlocks it. Decrypt with that and act seven's message is the flag.

FLAG `ICS{th3_53v3nth_3nc0r3_w45_n3v3r_h3r5}`

15. The Candidate Who Never Whispered

[stego / crypto]

Four files: a poster, a transcript, interview notes, and `private_broadcast.enc`. The trick is spread across the archive as instructions. The poster's illuminated paragraph is an acrostic — first letter of each of nine lines spells **FIBONACCI**. The interview says "just natural growth" (Fibonacci again) and "a woven boundary, like a ZigZag, exactly three times high" (Rail Fence, 3 rails). The transcript's episode ID `BRAASI-EL-634`, split by odd/even positions, spells `BASE64` and `RAIL3` — confirming the mechanism.

So: pull the characters at Fibonacci-numbered (1-based) positions out of the broadcast — that gives `F{305WLG47N1N1_03}A77_PR`, clearly a flag but transposed — then run it through a 3-rail Rail Fence decrypt to get `FLAG{4773N710N_15_POW3R}`. Swap the generic `FLAG` prefix for the event's `iCS` and that's it. (Body reads "ATTENTION IS POWER" in leet.)

→ Solver script: `candidate_never_whispered_solve.py`

FLAG `iCS{4773N710N_15_POW3R}`

16. The Impossible Meeting

[forensics / timezone logic]

A single `.ics` calendar. It defines three `VTIMEZONE` blocks with full DST rules, which is the puzzle shouting that timezone math is the whole game. The master event is `FREQ=DAILY;COUNT=8` with one `EXDATE`, leaving seven occurrences, and each survivor has an override that *relocates* it to a different timezone.

Two traps. First, each override has both a `RECURRENCE-ID` (which only *identifies* the original slot, always a flat 09:00) and a `DTSTART` (when it *actually* runs, with deliberate odd seconds) — you want the `DTSTART`. Second, DST: two Berlin events five days apart have different UTC offsets (CET +01:00 vs CEST +02:00 across the March 29 transition), and if you apply the same offset to both, one byte comes out as an unprintable control character and the key is silently wrong.

The description is the instruction sheet: sort the seven by UTC instant, take `epoch mod 256` from each — one byte per meeting. Done right, all seven land in printable ASCII and spell `q7M2vK9`, which is the checksum that you got it right. The attachment is a `IMG1\x 00` header, then repeating-XOR over a raw DEFLATE stream. XOR with the key and inflate — and note you need the streaming decompressor, because the stream has no final-block terminator and the one-shot `zlib.decompress` throws even with the correct key. That last bit genuinely cost me time.

→ Solver script: `impossible_meeting_solve.py`

FLAG `ICS{utc_t1m3_b34t5_w41l_c10ck5}`

17. The Document That Remembers

[forensics / CRDT]

A recovered collaborative editor that only shows the final clean copy, but legal needs the exact document state bound to receipt `hold-7f1c`. The audit log gives that receipt a causal frontier (`a17f:369`, `b03c:331`, `c92e:289`) and a SHA-256; the final snapshot uses a *later* frontier, so the published text isn't the target — I have to replay the operation journal up to the hold's frontier.

`editor/app.js` is the renderer. `operations.bin` starts with a SLATELOG2 signature and the rest is zlib-deflated JSON: insert/delete ops authored by three actors, with inserted characters XOR-decoded against a per-actor mask and a causal clock, deletes tombstoning nodes, and rendering walking the op tree from `root:0`. I reused the editor's own algorithm but stopped applying operations at the receipt frontier instead of the final one. The recovered text's SHA-256 matched the receipt exactly, and it carried the authorization token.

FLAG `ICS{t0mb5t0n35_r3m3mb3r_42b7}`

Scoreboard

For quick reference, here's everything in one place.

Challenge	Category	Flag
House of Mirrors	crypto	iCS{m1rr0r_m1rr0r_..._t0_411}
BOS-400 Telegraph	crypto	iCS{b0s400_d3c0mm1ss10n3d}
Sealed Train	crypto	iCS{POW3R_W45_1N_7H3_5P33CH}
The Seduction	audio stego	ICS{v3lv3t_st4t1c_h0ps_3142_7c9e}
Tea Party	forensics	ICS{W3LCOME_..._T34_P4RTY}
Chameleo Great Power	web	iCS{s3l3c7_qu3r13s_n33d_b0und4r13s}
Kotoamatsukami	web	ICS{7h3_d3c1510n_w45_n3v3r_y0ur5}
Vanishing Stars	cloud	ICS{7H3_5473LL173_N3V3R_F41L3D}
Man Behind the Mask	pwn	iCS{N3W_M45K_54M3_M3M0RY}
The Leader	reversing	ICS{1984_c0rrupt10n_1776_1337}
The Art of Programming	esolang	ICS{This_is_Mondarian_art}
Fairchild	stego	ICS{d2f8c2624be366cdfe3cee06dd216cc3}
Contaminated Container	devops	ICS{d3l3t3d_fr0m_v13w_n0t_fr0m_h15t0ry}
The Seventh Encore	supply chain	ICS{th3_53v3nth_3nc0r3_w45_n3v3r_h3r5}
Candidate Who Never Whispered	stego	iCS{4773N710N_15_POW3R}
The Impossible Meeting	forensics	ICS{utc_t1m3_b34t5_w411_c10ck5}
The Document That Remembers	forensics	ICS{t0mb5t0n35_r3m3mb3r_42b7}

All solver scripts referenced above live in the repo under `ALLWriteUps/scripts/` (or the linked path for the two that predate that folder). — P.M.